

# Software architecture:

## 1. Data Design:

- **Data Modeling:**

- Process of defining the structure, relationships, and constraints of data to be stored in a database.
- Techniques include Entity-Relationship Diagrams (ERD), UML diagrams, and schema definitions.

- **Data Structures:**

- Arrangement of data elements and their organization within memory or storage for efficient access and manipulation.
- Examples include arrays, linked lists, trees, graphs, and hash tables.

- **Databases and Data Warehouse:**

- **Databases:** Systems for storing, managing, and retrieving structured data.
- **Data Warehouse:** Repository for storing large volumes of historical, aggregated, and analytical data for decision-making purposes.

## 2. ASSESSING ALTERNATIVE ARCHITECTURAL DESIGNS

### An Architecture Trade-Off Analysis Method

The Software Engineering Institute (SEI) has developed an architecture trade-off analysis method (ATAM) that establishes an iterative evaluation process for software architectures. The design analysis activities that follow are performed iteratively:

1. **Collect scenarios.** A set of use cases is developed to represent the system from the user's point of view.
2. **Elicit requirements, constraints, and environment description.** This information is determined as part of requirements engineering and is used to be certain that all stakeholder concerns have been addressed.
3. **Describe the architectural styles/patterns that have been chosen to address the scenarios and requirements.** The architectural style(s) should be described using one of the following architectural views:
  - Module view for analysis of work assignments with components and the degree to which information hiding has been achieved.
  - Process view for analysis of system performance.
  - Data flow view for analysis of the degree to which the architecture meets functional requirements.

**4. Evaluate quality attributes by considering each attribute in isolation.**

The number of quality attributes chosen for analysis is a function of the time available for review and the degree to which quality attributes are relevant to the system at hand. Quality attributes for architectural design assessment include reliability, performance, security, maintainability, flexibility, testability, portability, reusability, and interoperability.

**5. Identify the sensitivity of quality attributes to various architectural attributes for a specific architectural style.** This can be accomplished by making small changes in the architecture and determining how sensitive a quality attribute, say performance, is to the change. Any attributes that are significantly affected by variation in the architecture are termed sensitivity points.

**6. Critique candidate architectures (developed in step 3) using the sensitivity analysis conducted in step 5.**

### 3. Architectural Complexity

A useful technique for assessing the overall complexity of a proposed architecture is to consider dependencies between components within the architecture. These dependencies are driven by information/control flow within the system. Zhao suggests three types of dependencies:

- **Sharing dependencies** represent dependence relationships among consumers who use the same resource or producers who produce for the same consumers
- **Flow dependencies** represent dependence relationships between producers and consumers of resources.
- **Constrained dependencies** represent constraints on the relative flow of control among a set of activities.

## 4. ARCHITECTURAL MAPPING USING DATA FLOW

A mapping technique, called structured design is often characterized as a data flow oriented design method because it provides a convenient transition from a data flow diagram to software architecture. The transition from information flow (represented as a DFD) to program structure is accomplished as part of a six step

process:

1. the type of information flow is established,
2. flow boundaries are indicated,
3. the DFD is mapped into the program structure,
4. control hierarchy is defined,
5. the resultant structure is refined using design measures and heuristics, and
6. the architectural description is refined and elaborated.

In order to perform the mapping, the type of information flow must be determined. One type of information flow is called **transform flow** and exhibits a linear quality. Data flows into the system along an **incoming flow** path where it is transformed from an external world representation into internalized form.

Once it has been internalized, it is processed at a **transform center**. Finally, it flows out of the system along an **outgoing flow path** that transforms the data into external world.

# Architectural Mapping

- Structured design provides a convenient transition from a data flow diagram to software Architecture

## Types of information flow

The 2 different types of information flows:

1. **transaction flow** - a single data item triggers information flow along one of many paths
2. **transform flow**
  - overall data flow is sequential and flows along a small number of straight line paths
    - *Incoming Flow:* The paths that transform the external data into an internal form
    - *Transform Center:* The incoming data are passed through a transform center and begin to move along paths that lead it out of the software
    - *Outgoing Flow:* The paths that move the data out of the software

## Software Engineering-Mapping Requirements in Software Architecture:

Software requirements can be mapped into various representations of the design model. The architectural styles represent radically different architectures, so it should come as no surprise that a comprehensive mapping that accomplishes the transition from the requirements model to a variety of architectural styles does not exist. In fact, there is no practical mapping for some architectural styles, and the designer must approach the translation of requirements to design for these styles in an ad hoc fashion.

To illustrate one approach to architectural mapping, we consider the call and return architecture—an extremely common structure for many types of systems. The mapping technique to be presented enables a designer to derive reasonably complex call and return architectures from data flow diagrams within the requirements model. The technique, sometimes called structured design, has its origins in earlier design concepts that stressed modularity , top-down design , and structured programming . Stevens, Myers, and Constantine were early proponents of software design based on the flow of data through a system. Early work was refined and presented in books by Myers and Yourdon and Constantine.

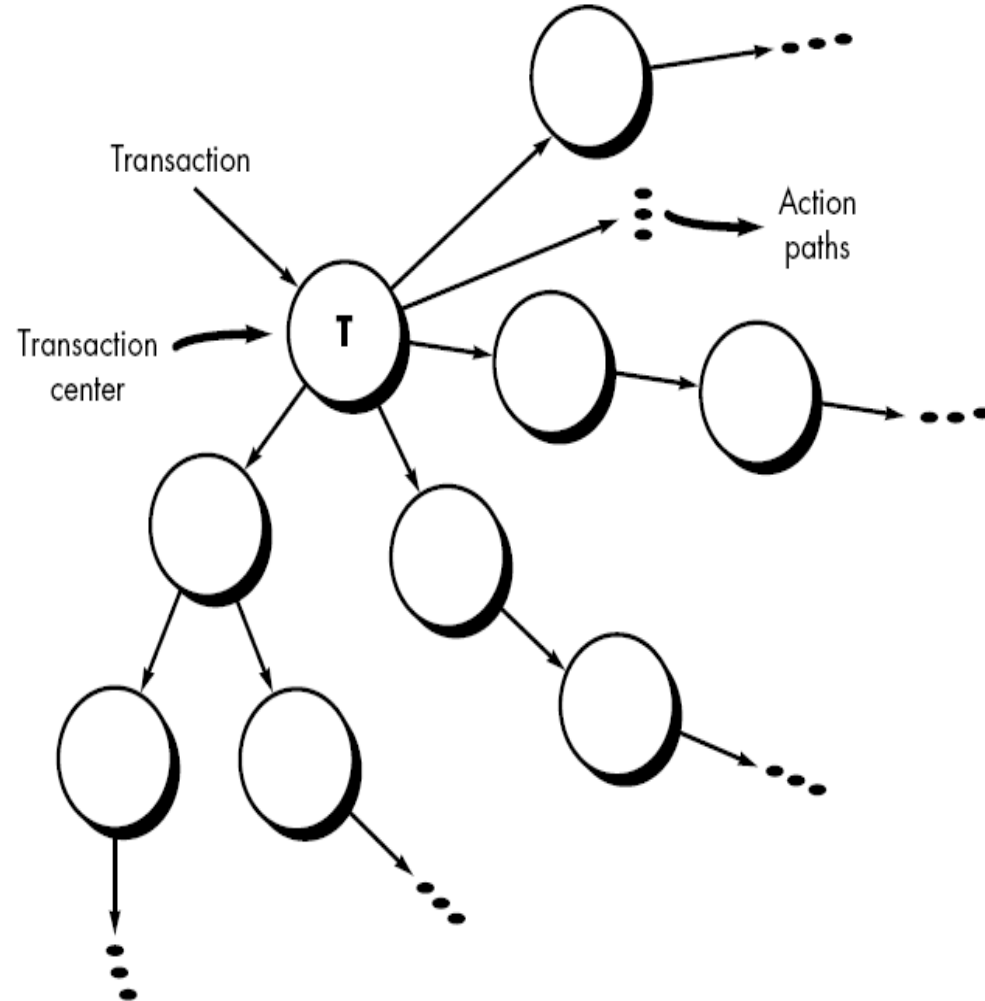
Structured design is often characterized as a data flow-oriented design method because it provides a convenient transition from a data flow diagram to software architecture. The transition from information flow (represented as a DFD) to program structure is accomplished as part of a six-step process: **(1)** the type of information flow is established; **(2)** flow boundaries are indicated; **(3)** the DFD is mapped into program structure; **(4)** control hierarchy is defined; **(5)** resultant structure is refined using design measures and heuristics; and **(6)** the architectural description is refined and elaborated.

The type of information flow is the driver for the mapping approach required in step 3. In the following sections we examine two flow types.

- **Transform Flow**
- **Transaction Flow**

# Transform Flow

Recalling the fundamental system model (level 0 data flow diagram), information must enter and exit software in an "external world" form. For example, data typed on a keyboard, tones on a telephone line, and video images in a multimedia application are all forms of external world information. Such externalized data must be converted into an internal form for processing. Information enters the system along paths that transform external data into an internal form. These paths are identified as incoming flow. At the kernel of the software, a transition occurs. Incoming data are passed through a transform center and begin to move along paths that now lead "out" of the software. Data moving along these paths are called outgoing flow. The overall flow of data occurs in a sequential manner and follows one, or only a few, "straight line" paths. When a segment of a data flow diagram exhibits these characteristics, transform flow is present.



## Transaction Flow

- The fundamental system model implies transform flow; therefore, it is possible to characterize all data flow in this category. However, information flow is often characterized by a single data item, called a transaction, that triggers other data flow along one of many paths. When a DFD takes the form shown in figure, transaction flow is present.
- Transaction flow is characterized by data moving along an incoming path that converts external world information into a transaction. The transaction is evaluated and based on its value, flow along one of many action paths is initiated. The hub of information flow from which many action paths emanate is called a transaction center.
- It should be noted that, within a DFD for a large system, both transform and transaction flow may be present. For example, in a transaction-oriented flow, information flow along an action path may have transform flow characteristics.